

Теория параллелизма

Отчет

Задание №6-8

Выполнил, Пономарев Андрей Александрович 23934
25.05.2025

Цель работы: реализовать программу для решения уравнения теплопроводности, изучить время выполнения программы на CPU на одном ядре и на нескольких, изучить время выполнения на GPU, выполнить профилирование программы с использованием “Nsight Systems”.

Используемый компилятор: gpc++

Используемый профилировщик: Nsight Systems

Замер времени: библиотека chrono

Выполнение на CPU

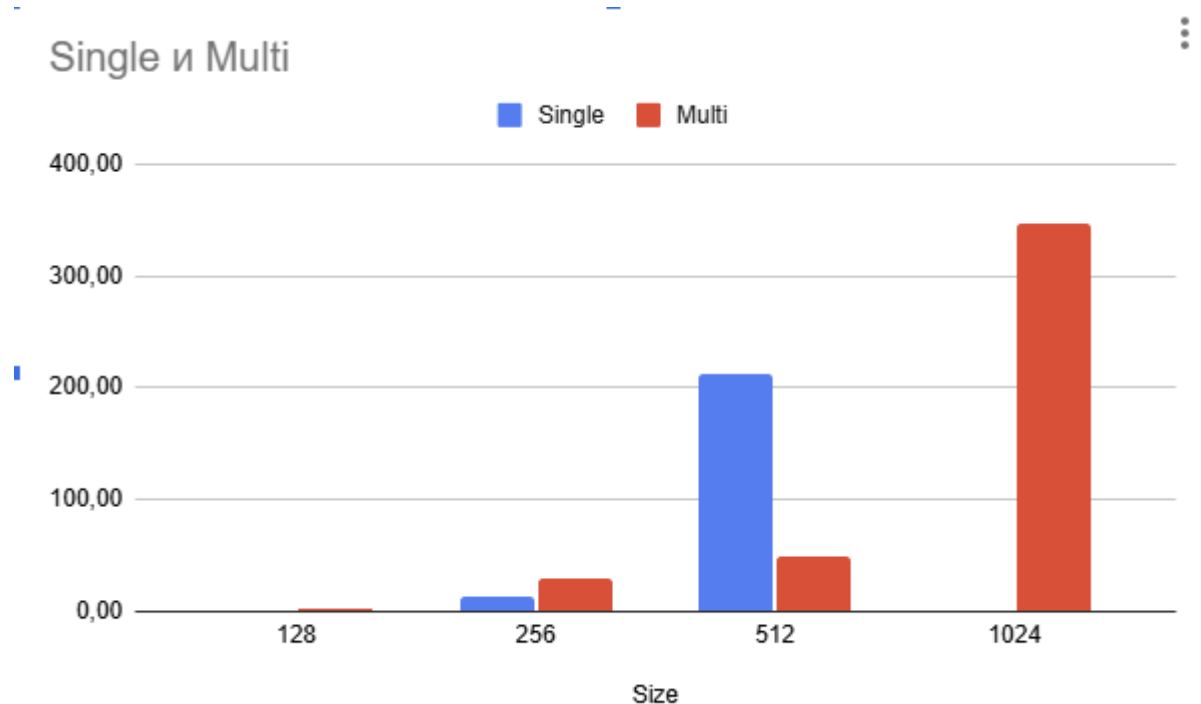
CPU-onecore

Размер сетки	Время выполнения	Точность (final error)	Количество итераций
128*128	0.920166	0.000001	30075
256*256	13.858765	0.000001	102885
512*512	211.502202	0.000001	339599

CPU-multicore

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	2.446336	0.000001	30075
256*256	29.769453	0.000001	102885
512*512	49.666926	0.000001	339599
1024*1024	346.737076	0.000001	1000000

Диаграмма сравнения время работы CPU-one и CPU-multi



Выполнение на GPU

Этапы оптимизации на сетке 512*512

Этап №	Время выполнения	Точность	Максимальное количество итераций	Комментарии
1	15.71309	0.000001	1000000	первая версия
2	15.631527	0.000001	1000000	попытка внедрить collapse()
3	12.019444	0.000001	1000000	error считается не на каждом шаге

Начальный вариант:

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
98.1	503252831	1	503252831.0	503252831.0	503252831	503252831	0.0	PushPop	init
1.0	4876794	1	4876794.0	4876794.0	4876794	4876794	0.0	PushPop	while
0.5	2546345	100	25463.5	22427.5	21609	258820	23632.4	PushPop	calc
0.4	2276244	100	22762.4	22147.0	21353	30312	1449.1	PushPop	copy

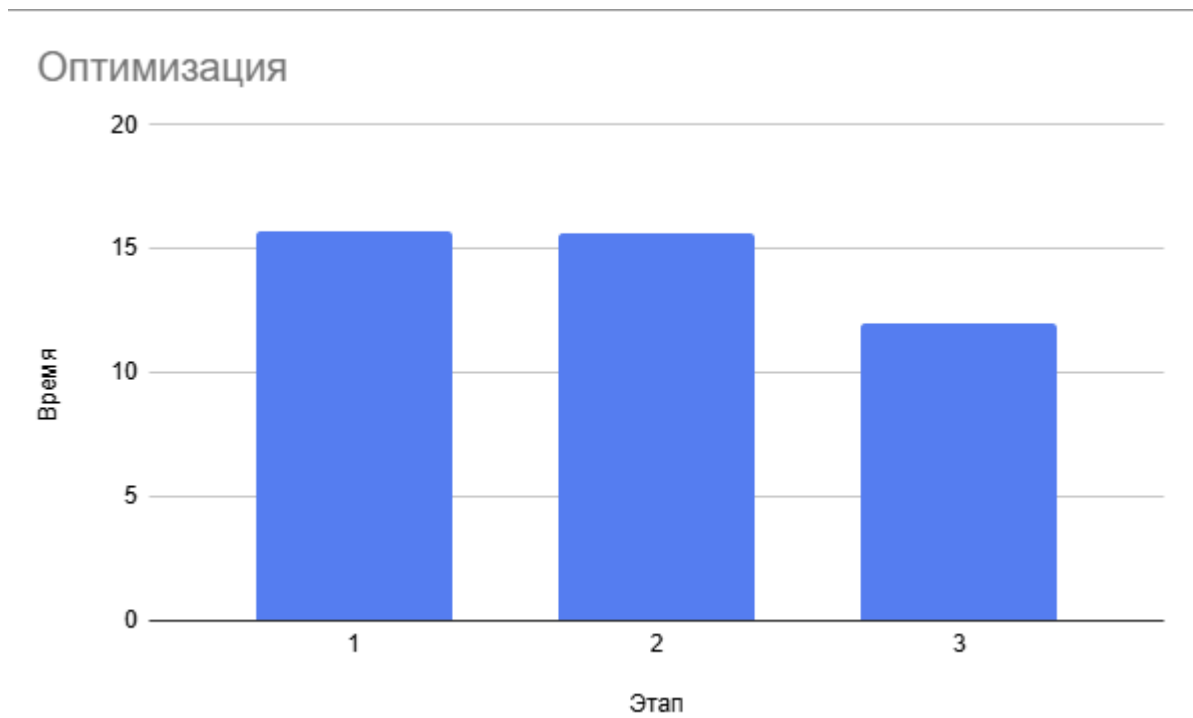
Второй этап: внедрение collapse():

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
98.3	500165503	1	500165503.0	500165503.0	500165503	500165503	0.0	PushPop	init
0.8	4267137	1	4267137.0	4267137.0	4267137	4267137	0.0	PushPop	while
0.4	2219715	100	22197.2	20084.5	19484	215165	19509.9	PushPop	calc
0.4	2001112	100	20011.1	19841.5	19366	30053	1104.4	PushPop	copy

Третий этап: Проверка ошибки каждую 1000 итерацию

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
98.5	565215828	1	565215828.0	565215828.0	565215828	565215828	0.0	PushPop	init
0.8	4304648	1	4304648.0	4304648.0	4304648	4304648	0.0	PushPop	while
0.4	2249081	100	22490.8	19989.5	19233	251441	23162.8	PushPop	calc
0.3	2004417	100	20044.2	19669.0	18857	30433	1707.1	PushPop	copy

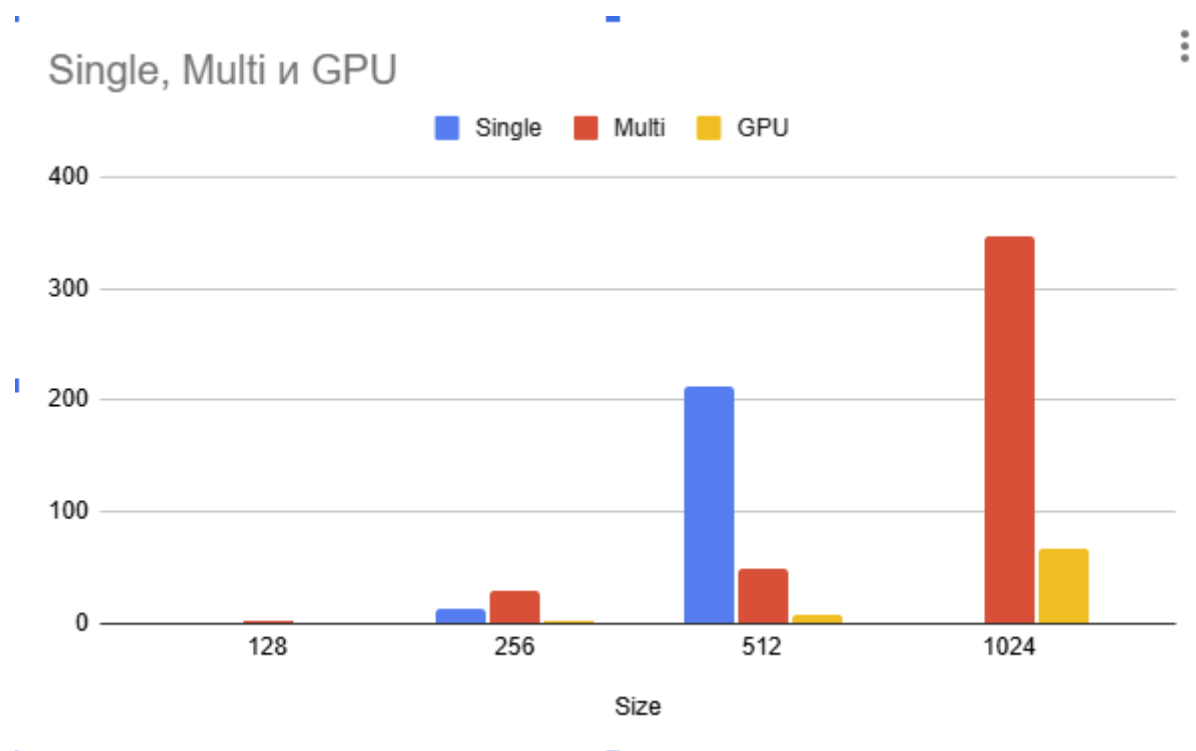
Диаграмма оптимизации



GPU – оптимизированный вариант

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	0.647406	0.000001	31000
256*256	2.290811	0.000001	103000
512*512	8.616552	0.000001	340000
1024*1024	66.571557	0.000001	999999

Диаграмма сравнения времени работы CPU-one, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток



Выполнение на GPU с cuBLAS

Оптимизация на GPU с cuBLAS

Этап №	Время выполнения	Точность	Максимальное количество итераций	Комментарии
1	8.56879	9.92418e-07	1000000	первая версия
2	12.1728	13.5956	1000000	упрощенное копирование
3	7.85133	9.92418e-07	1000000	объединение вычисления новой сетки и расчета ошибки

Первый этап: изначальная версия

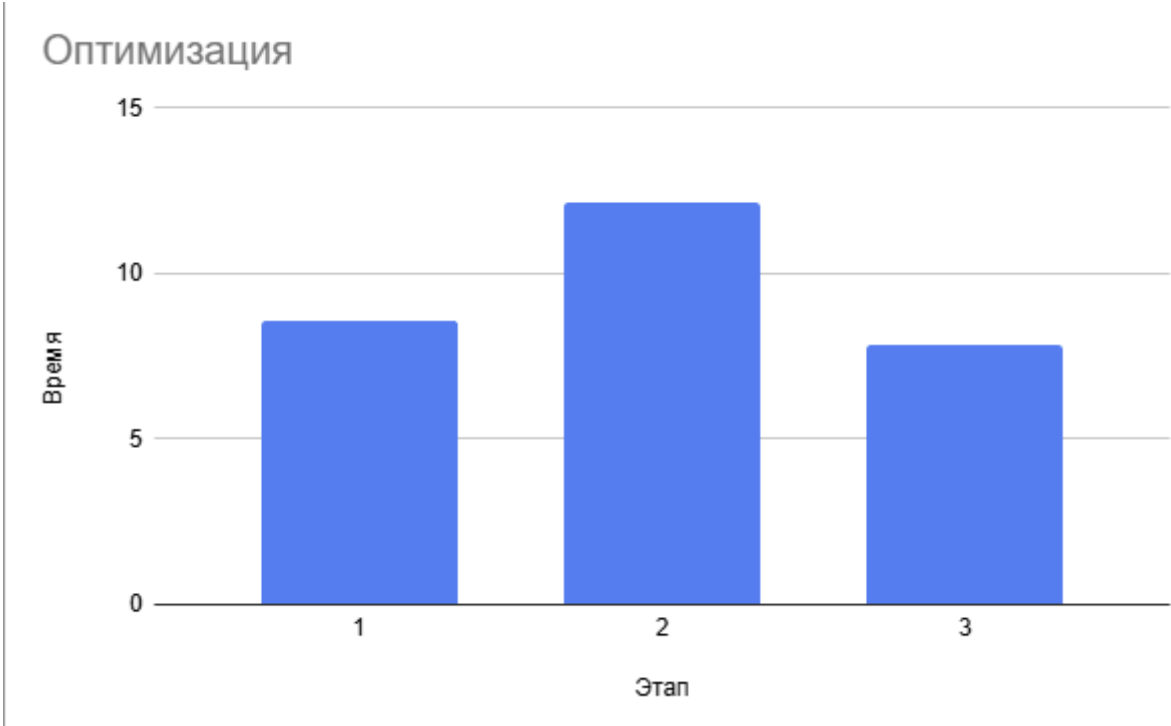
Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
49.4	16825456	1	16825456.0	16825456.0	16825456	16825456	0.0	PushPop	SolverLoop
24.3	8264112	1	8264112.0	8264112.0	8264112	8264112	0.0	PushPop	InitGrid
12.0	4081851	100	40818.5	39203.0	38219	145446	10878.1	PushPop	Compute
11.6	3957627	100	39576.3	39239.0	37754	50814	1786.8	PushPop	CopyBack
2.6	901194	1	901194.0	901194.0	901194	901194	0.0	PushPop	ErrorCalc

Второй этап: упрощенное копирование

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
48.4	7452367	1	7452367.0	7452367.0	7452367	7452367	0.0	PushPop	SolverLoop
17.9	2751055	1	2751055.0	2751055.0	2751055	2751055	0.0	PushPop	InitGrid
14.0	2149785	100	21497.9	19933.5	19115	162116	14310.3	PushPop	Compute
13.0	2003924	100	20039.2	19603.5	18777	37111	2355.3	PushPop	CopyBack
6.8	1053308	1	1053308.0	1053308.0	1053308	1053308	0.0	PushPop	ErrorCalc

Третий этап: объединение вычисления новой сетки и расчета ошибки

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
51.9	8057132	1	8057132.0	8057132.0	8057132	8057132	0.0	PushPop	SolverLoop
14.7	2279324	1	2279324.0	2279324.0	2279324	2279324	0.0	PushPop	InitGrid
14.1	2196959	100	21969.6	20337.5	19699	122652	10465.9	PushPop	Compute+Error
13.0	2017005	100	20170.1	19503.0	18938	39194	2834.8	PushPop	CopyBack
6.4	988261	1	988261.0	988261.0	988261	988261	0.0	PushPop	ErrorReduce



оптимизированный вариант

Размер сетки	Время выполнения	Точность	Количество итераций
128	0.599365	8.77535e-07	30501
256	2.2255	9.91175e-07	103001
512	7.85133	9.92418e-07	340001
1024	68.9399	1.37251e-06	1000000

Выполнение на GPU с CUB

Этап №	Время выполнения	Точность	Максимальное количество итераций	Комментарии
1	19.4416	9.92418e-07	1000000	первая версия
2	16.6524	9.99985e-07	1000000	замена cudaMalloc на CUB
3	22.4842	10	1000000	захват CUDA Graph до цикла

Первый этап: изначальная версия

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
95.6	289212700	3	96404233.3	229370.0	228246	288755084	166580723.1	cudaMalloc
1.1	3263335	102	31993.5	17775.5	16528	748637	94977.6	cudaMemcpy
0.7	2161323	100	21613.2	18313.5	15588	98472	11165.3	cudaGraphInstantiate_v12000
0.7	2009087	200	10045.4	7521.0	1660	886869	62505.4	cudaLaunchKernel
0.4	1357774	100	13577.7	13609.5	12426	15618	651.3	cudaStreamSynchronize
0.4	1074626	100	10746.3	9958.5	9067	30481	2872.9	cudaGraphLaunch_v10000
0.3	1025138	100	10251.4	9574.5	8181	20701	2193.1	cudaGraphExecDestroy_v10000
0.2	685101	3	228367.0	188828.0	185419	310854	71456.2	cudaFree
0.2	512427	100	5124.3	4944.0	4060	9571	663.8	cudaStreamDestroy
0.2	477990	100	4779.9	4162.0	3575	28642	2945.3	cudaStreamCreate
0.1	364769	100	3647.7	3094.5	2682	39521	3675.6	cudaStreamBeginCapture_v10000
0.1	166350	100	1663.5	1629.5	1394	2801	161.1	cudaGraphDestroy_v10000
0.0	131529	100	1315.3	1249.5	977	4629	358.0	cudaStreamEndCapture_v10000
0.0	8385	1	8385.0	8385.0	8385	8385	0.0	cudaDeviceSynchronize
0.0	1376	1	1376.0	1376.0	1376	1376	0.0	cuModuleGetLoadingMode

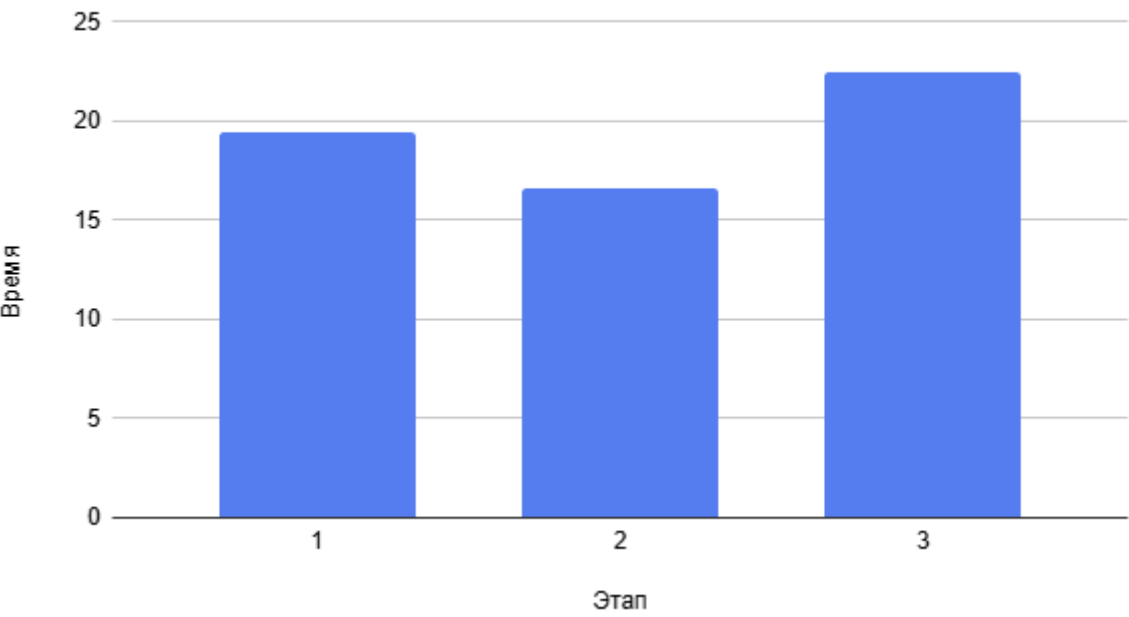
Второй этап: замена cudaMalloc на CUB

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
95.8	194139687	3	64713229.0	154904.0	153820	193830963	111819237.7	cudaMalloc
1.3	2607909	102	25567.7	14163.5	13747	589725	79277.7	cudaMemcpy
0.7	1374664	200	6873.3	4578.0	987	741755	52294.7	cudaLaunchKernel
0.6	1257434	100	12574.3	12460.5	10254	24231	1370.8	cudaStreamSynchronize
0.6	1169117	100	11691.2	9438.0	8208	63666	7069.0	cudaGraphInstantiate_v12000
0.3	631584	100	6315.8	6121.5	5576	17305	1221.4	cudaGraphLaunch_v10000
0.3	571838	100	5718.4	5412.5	5148	22529	1837.6	cudaGraphExecDestroy_v10000
0.2	312473	100	3124.7	2909.5	2758	19572	1675.0	cudaStreamDestroy
0.1	259085	100	2590.9	2336.0	2108	18340	1623.0	cudaStreamCreate
0.1	193777	100	1937.8	1668.5	1521	23534	2191.1	cudaStreamBeginCapture_v10000
0.0	83204	100	832.0	823.5	777	1124	51.9	cudaGraphDestroy_v10000
0.0	64615	100	646.2	616.5	561	3165	257.1	cudaStreamEndCapture_v10000
0.0	22226	3	7408.7	1385.0	1149	19692	10638.3	cudaEventCreateWithFlags
0.0	8641	1	8641.0	8641.0	8641	8641	0.0	cudaDeviceSynchronize
0.0	5857	3	1952.3	882.0	840	4135	1890.4	cudaEventRecord
0.0	906	1	906.0	906.0	906	906	0.0	cuModuleGetLoadingMode

Третий этап: захват CUDA Graph до цикла

Time (%)	Total Time (ns)	Num Calls	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
97.0	181152680	3	60384226.7	140234.0	139415	180873031	104346365.4	cudaMalloc
0.8	1404931	200	7024.7	6912.0	1520	18253	5471.0	cudaStreamSynchronize
0.7	1262273	100	12622.7	12190.0	11866	40499	3010.0	cudaMemcpyAsync
0.7	1224486	101	12123.6	3574.0	3426	810480	80311.1	cudaLaunchKernel
0.6	1062668	2	531334.0	531334.0	492631	570037	54734.3	cudaMemcpy
0.2	390754	100	3907.5	3517.0	3366	24814	2175.9	cudaGraphLaunch_v10000
0.1	93716	1	93716.0	93716.0	93716	93716	0.0	cudaGraphInstantiate_v12000
0.0	35634	1	35634.0	35634.0	35634	35634	0.0	cudaStreamBeginCapture_v10000
0.0	18792	1	18792.0	18792.0	18792	18792	0.0	cudaStreamCreate
0.0	14971	1	14971.0	14971.0	14971	14971	0.0	cudaGraphExecDestroy_v10000
0.0	11716	3	3905.3	1213.0	1090	9413	4770.2	cudaEventCreateWithFlags
0.0	6781	3	2260.3	1255.0	695	4831	2243.8	cudaEventRecord
0.0	5501	1	5501.0	5501.0	5501	5501	0.0	cudaDeviceSynchronize
0.0	3874	1	3874.0	3874.0	3874	3874	0.0	cudaStreamEndCapture_v10000
0.0	3810	1	3810.0	3810.0	3810	3810	0.0	cudaStreamDestroy
0.0	1517	1	1517.0	1517.0	1517	1517	0.0	cuModuleGetLoadingMode
0.0	1292	1	1292.0	1292.0	1292	1292	0.0	cudaGraphDestroy_v10000

Оптимизация CUB

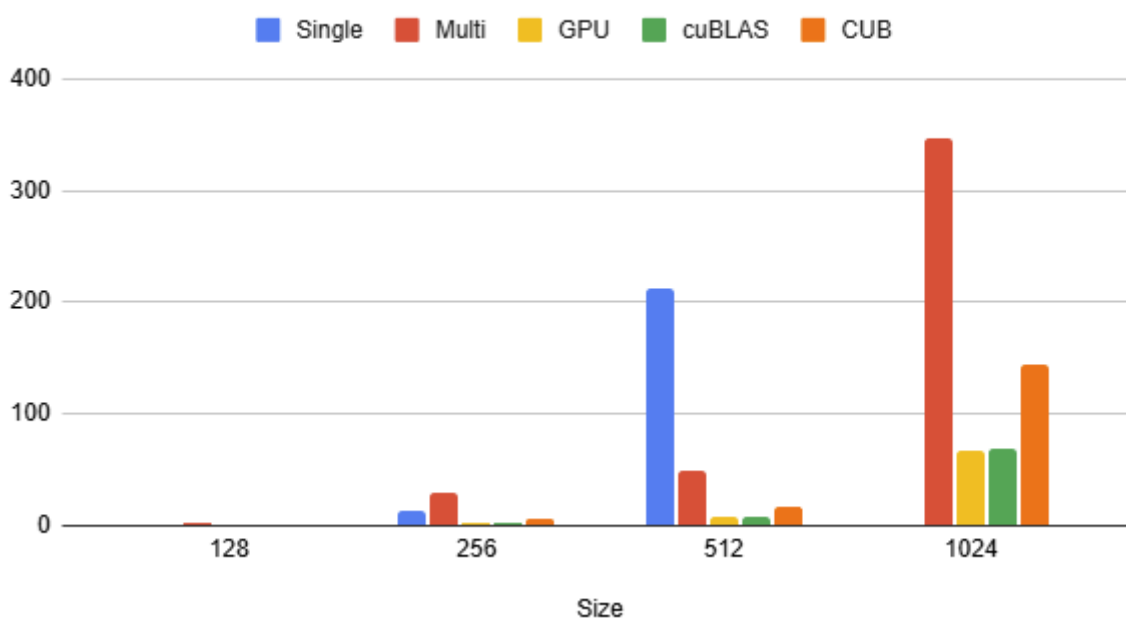


оптимизированный вариант

Размер сетки	Время выполнения	Точность	Количество итераций
128	1.59831	9.99933e-07	36682
256	6.75716	9.91175e-07	124931
512	16.6524	9.92418e-07	409740
1024	144.691	1.37251e-06	1000000

Сравнения всех вариантов

Single, Multi, GPU, cuBLAS и CUB



Вывод

На итоговой диаграмме видно, что распараллеливание на GPU работает быстрее всего. Большое количество физических ядер в совокупности большим объёмом видеопамати позволяют добиться реального распределения работы в этой задаче. Также хорошо видно, что чем больше сетка, тем быстрее работает GPU по сравнению с остальными. Это объясняется тем, что данные на GPU сразу уходят на обработку, а на CPU необходимо ожидать “своей очереди”. На небольших же значениях разрыв меньше. Это объясняется тем, что хотя процессор имеет небольшое количество ядер, его хватает для распараллеливания, соответственно время “ожидания в очереди” сокращается. К тому же процессорные ядра быстрее, соответственно если данных на обработку не так много, процессор в некоторых задачах может даже обгонять видеокарту.

Дополнение про cuBLAS: видно, что библиотека не слишком смогла помочь ускорению. С другой стороны получилось добиться весьма низких показателей ошибки.

Дополнение про CUB: видно, что реализация через cuda в нашей задаче не дала значительного выигрыша по сравнению с cuBLAS. Возможно это вызвано тем, что на небольших сетках частое обращение к памяти съедает весь прирост производительности.